Thesis Notebook

2D Orbit with the suns density profile and mass function, along with dissipative forces

Setting functions up and choosing variables

In[∘]:=
```
(*R values in Schwarzschild coordinates*)
RStar = 8;(*Radius of the Star*)
M = 1;
```

Exterior Schwarzschild Isotropic coordinates definitions $r(\bar{r})$, $\bar{r}(r)$, $A(\bar{r})$, $\alpha(\bar{r})$

$r(\bar{r})$, $\bar{r}(r)$

In[∘]:=
```
rbarofrOutside[r_] := 1/2 (-M + r + Sqrt[r^2 - 2 M r]);
```

In[∘]:=
```
rbarStar = rbarofrOutside[RStar];
```

In[∘]:=
```
rofrbarOutside[rbar_] := rbar (1 + M/(2 rbar))^2;
```

$A(\bar{r})$, $\alpha(\bar{r})$

In[∘]:=
```
A[rbar_] := (1 + M/(2 rbar))^2;
α[rbar_] := (1 - M/(2 rbar))/(1 + M/(2 rbar));
```

Mass and Density functions from 2405.08113

Fixing the rho0 by letting the total mass integral equal to 1

In[∘]:=
```
rho[r_] := rho0 (1 - r/R)^6;
(*massIntegral =  4 Pi Integrate[rho[r] r^2, {r, 0, R}]; Set equal to 1*)
```

In[∘]:=
```
rhoCentre[R_] := 63/(Pi R^3) (*From the above*)
rho0 = rhoCentre[RStar];
```

Mass function

In[∘]:=
```
MassFunc[r_] := 252 (r^9/(9 RStar^9) - 3 r^8/(4 RStar^8) + 15 r^7/(7 RStar^7) - 10 r^6/(3 RStar^6) + 3 r^5/RStar^5 - 3 r^4/(2 RStar^4) + r^3/(3 RStar^3))
```

Density function

In[∘]:=
```
rho[r_] := rho0 (1 - r/RStar)^6
```

Numerically integrating for the pressure and the phi in α = exp(phi)

Defining the ODEs

In[∘]:=
```
(*TOV differential equation*)
dPdr[r_, Pressure_] := -(rho[r] + Pressure) ((MassFunc[r] + 4 Pi Pressure r^3)/(r (r - 2 MassFunc[r])));
```

In[∘]:=
```
(*Phi differential equation for metric*)
dPhidrMetric[r_, Pressure_] := ((MassFunc[r] + 4 Pi Pressure r^3)/(r (r - 2 MassFunc[r])));
```

Numerically integrating these coupled ODEs

In[∘]:=
```
rmin = 10^-16 (*For these integrations*);
```

In[∘]:=
```
eq1 = P'[r] == dPdr[r, P[r]];
eq2 = Phi'[r] == dPhidrMetric[r, P[r]];
BCs = {P[RStar] == 0, Phi[RStar] == 1/2 Log[1 - 2 M/RStar]};
(*Phi[RStar] from the Schwarzschild metric*)
```

In[∘]:=
```
sol1 = NDSolve[
    {eq1,
     eq2,
     BCs},
    {P, Phi},
    {r, RStar, rmin},
    WorkingPrecision → 40
   ][[1]]; (*Very small rmin to avoid singularity sarp mentioned*)
```

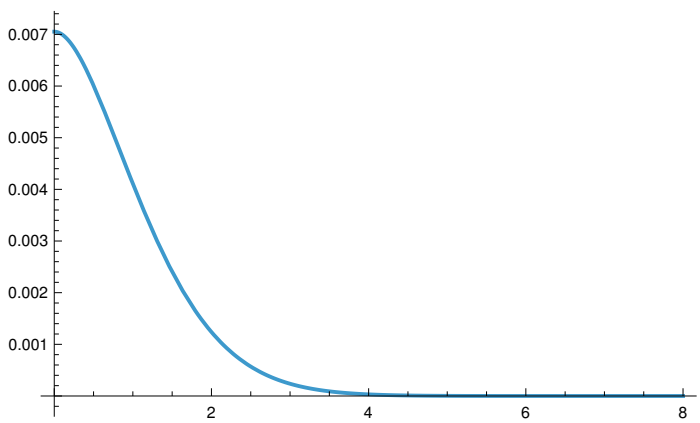
In[∘]:=
```
Pressure[r_] := P[r] /. sol1

PhiMetric[r_] = Phi[r] /. sol1;

(*Plot[Derivative[1][PhiMetric][r], {r, 0, RStar}]*)
(*Causing the issue it seems, but we have this Analytically!*)
```

In[∘]:=
```
Plot[Pressure[r], {r, 0, RStar}, PlotRange → All]
```

Out[∘]=



Solving for $\bar{r}(r)$ and $r(\bar{r})$ inside

Defining the ODE

In[∘]:=
```
drbardr[rbar_, r_] := rbar/r (1 - 2 MassFunc[r]/r)^(-1/2)
```

Numerically Integrating the above equation

In[∘]:=
```
eq = rbarofr'[r] == drbardr[rbarofr[r], r];
BC = {rbarofr[RStar] == rbarStar};
```

In[∘]:=
```
sol2 = NDSolve[
    {eq,
     BC},
    {rbarofr},
    {r, RStar, rmin},
    WorkingPrecision → 40
   ][[1]];
```

In[∘]:=
```
rbarofrInside[r_] = rbarofr[r] /. sol2;
```

Some sanity checks

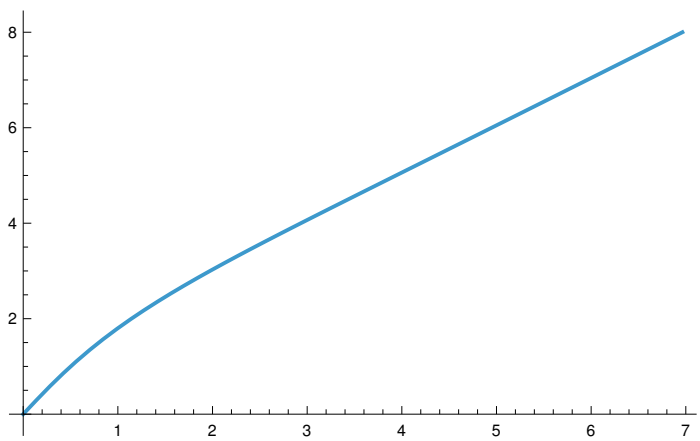
Now using the interpolation trick to obtain $r(\bar{r})$:

In[∘]:=
```
(*Table[ expression, {variable, start, end, step}*)
data = Prepend[Table[
    {rbarofrInside[r], r},
    {r, rmin, RStar, (RStar - rmin)/10000}(*10000 points and A is not messy*)
   ], {0, 0}];
```

In[∘]:=
```
rofrbarInside = Interpolation[data];
```

In[∘]:=
```
Plot[rofrbarInside[rbar], {rbar, 0, rbarStar}]
```

Out[∘]=



Finding the interior metric functions

Getting αInside($\bar{r}$):

In[∘]:=
```
αInside[rbar_?NumericQ] := Exp[PhiMetric[rofrbarInside[rbar]]]
```

Sanity checks

Making the derivative of alpha continuous

Interior derivative in terms of r

Doing things in r first as this is more analytic

In[∘]:=
```
dPhidrMetric[r_] := (MassFunc[r] + 4 Pi Pressure[r] r^3 / (r (r - 2 MassFunc[r])))
```

$$\text{dPhidrMetric}[r_] := \left(\frac{\text{MassFunc}[r] + 4\,\text{Pi}\,\text{Pressure}[r]\,r^3}{r\,(r - 2\,\text{MassFunc}[r])}\right)$$

In[∘]:=
```
alpha[r_] := Exp[PhiMetric[r]]
DERalphaIns[r_] := dPhidrMetric[r] × alpha[r] (*Analytic times numerical integral*)
```

Exterior derivative

In[∘]:=
```
DERαOutsiderbardummy = Derivative[1][α][rbar];
```

$$\text{DER}\alpha\text{Outsiderbar}[r_] := \left(\frac{1 - \frac{1}{2\,\text{rbar}}}{2\left(1 + \frac{1}{2\,\text{rbar}}\right)^2 \text{rbar}^2} + \frac{1}{2\left(1 + \frac{1}{2\,\text{rbar}}\right)\text{rbar}^2}\right) /.\ \text{rbar} \to \text{rbarofrOutside}[r]$$

```
(*More accurate subbing in after for rbar maybe?*)
```

$$\text{drbardr}[r_] := \frac{\text{rbarofrOutside}[r]}{r}\left(1 - \frac{2\,M}{r}\right)^{-\frac{1}{2}}$$

In[∘]:=
```
DERalpha[r_] := drbardr[r] × DERαOutsiderbar[r] (*Chain rule*)
```

Checking to see if it worked

Numbers at star boundary

In[∘]:=
```
DERalpha[RStar] // N
DERalphaIns[RStar] // N
```

Out[∘]= 0.0180422

Out[∘]= 0.0180422

In[∘]:=
$$\text{diff} = \text{Abs}\left[1 - \frac{\text{DERalphaIns}[\text{rbarofrInside}[\text{RStar}]]}{\text{DERalpha}[\text{rbarofrOutside}[\text{RStar}]]}\right]$$

Out[∘]= 0.000020941087477202766822069477603 93

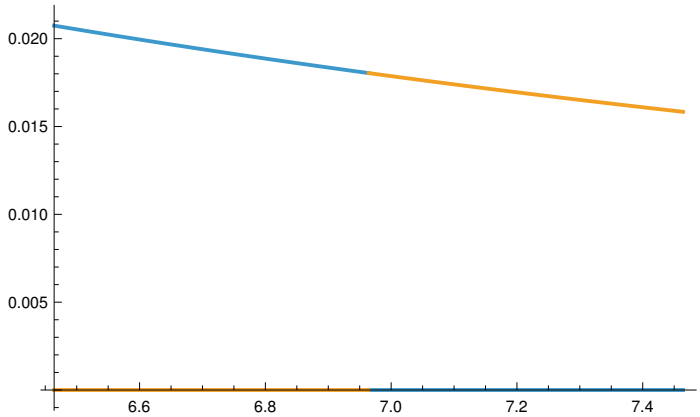
In[∘]:=
```
Abs[Log10[diff]] (*Amount of digits in agreement*)
```

Out[∘]= 4.679000769023691168696772066335 1

Plots at star boundary

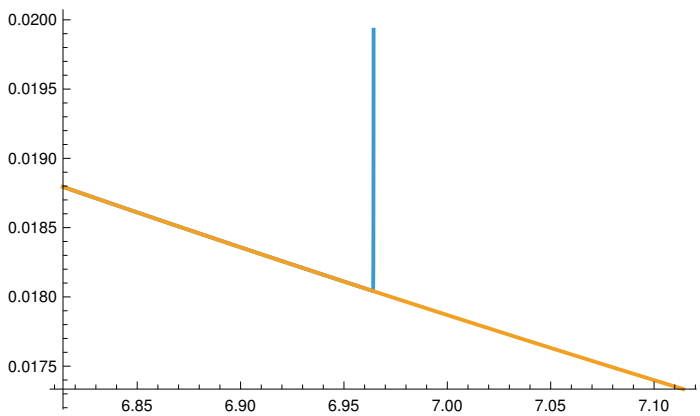
In[∘]:=
```
Plot[{Piecewise[{{DERalphaIns[rofrbarInside[rbar]], rbar ≤ rbarStar}}],
  Piecewise[{{DERalpha[rofrbarOutside[rbar]], rbar ≥ rbarStar}}]},
 {rbar, rbarStar - 0.5, rbarStar + 0.5}]
```

Out[∘]=



In[∘]:=
```
Plot[{DERalphaIns[rofrbarInside[rbar]], DERalpha[rofrbarOutside[rbar]]},
 {rbar, rbarStar - 0.15, rbarStar + 0.15}]
```

Out[∘]=



Proper definitions in isotropic coords now

In[∘]:=
```
DERαOutside[rbar_] := Module[{r = rofrbarOutside[rbar]}, drbardr[r] × DERαOutsiderbar[r]]

DERαInside[rbar_] := Module[{r = rofrbarInside[rbar]},
  dPhidrMetric[r] × alpha[r]]
```

In[∘]:=
```
DERαFull[rbar_] := Piecewise[
  {
   {DERαInside[rbar], rbar < rbarStar},
   {DERαOutside[rbar], rbar ≥ rbarStar}
  }
 ]
```

In[∘]:=
```
DERαInside[rbarStar]
```

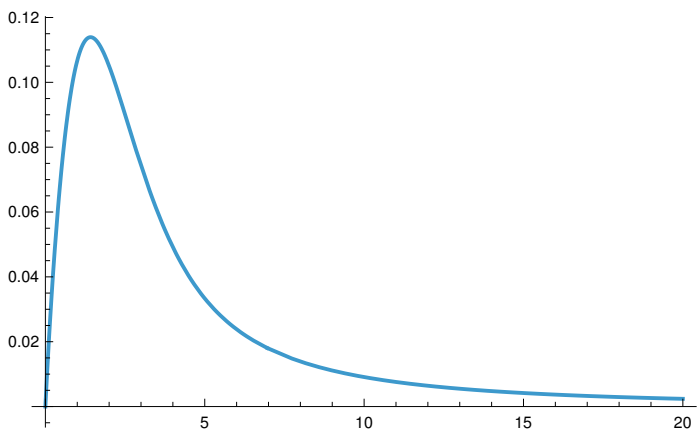
Out[∘]= 0.018042195912175805140910899390686170 5

In[∘]:=
```
DERαOutside[rbarStar];
```

In[∘]:=
```
(*DERαFull[rbar_]:=If[rbar<rbarStar,DERαInside[rbar],DERαOutside[rbar]]*)
```

In[∘]:=
```
Plot[DERαFull[rbar], {rbar, 0, 20}]
```

Out[∘]=

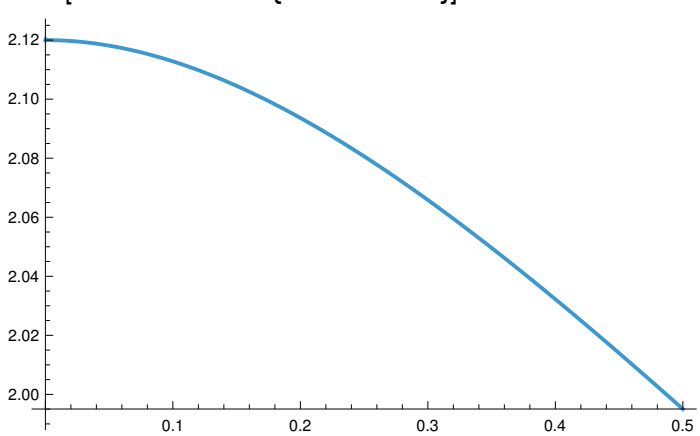


Going to do the same derivative setup as the α component for the A component

In[∘]:=
```
AInside[rbar_?NumericQ] := rofrbarInside[rbar]/rbar;
```

In[∘]:=
```
Plot[AInside[rbar], {rbar, 0, .5}]
```

Out[∘]=



In[∘]:=
```
A[rbarStar]
AInside[rbarStar]
```

Out[∘]= $\left(1+\frac{1}{7+4\sqrt{3}}\right)^2$

Out[∘]= 1.1487483155918532142434341436241685156 60

In[∘]:=
```
AFull[rbar_] := Piecewise[
   {
    {A[rbar], rbar ≥ rbarStar},
    {AInside[rbar], rbar < rbarStar}
   }
  ];
(*AFull[rbar_]:=If[rbar<rbarStar,AInside[rbar],A[rbar]]*)
```

In[∘]:= $\text{drbardr}[\text{rbar_}, \text{r_}] := \frac{\text{rbar}}{\text{r}}\left(1-\frac{2\,\text{MassFunc}[\text{r}]}{\text{r}}\right)^{-\frac{1}{2}}$

$\text{drdrbar}[\text{rbar_}] := \frac{\text{rofrbarInside}[\text{rbar}]}{\text{rbar}}\left(1-\frac{2\,\text{MassFunc}[\text{rofrbarInside}[\text{rbar}]]}{\text{rofrbarInside}[\text{rbar}]}\right)^{\frac{1}{2}}$

In[∘]:= $\text{DERAInside}[\text{rbar_}] := \frac{\text{drdrbar}[\text{rbar}]}{\text{rbar}} - \frac{\text{rofrbarInside}[\text{rbar}]}{\text{rbar}^2};$

```
DERAOutside[rbar_] := Derivative[1][A][rbar]
```

In[∘]:=
```
DERAFull[rbar_] := Piecewise[
   {
    {DERAOutside[rbar], rbar ≥ rbarStar},
    {DERAInside[rbar], rbar < rbarStar}
   }
  ];
(*DERAFull[rbar_]:=If[rbar<rbarStar,DERAInside[rbar],DERAOutside[rbar]]*)
```

In[∘]:=
```
DERAOutside[rbarStar]
DERAInside[rbarStar]
```

Out[∘]= $-\frac{4\left(1+\frac{1}{7+4\sqrt{3}}\right)}{\left(7+4\sqrt{3}\right)^2}$

Out[∘]= -0.022099489674330444219434004362804943 3

Density and Pressure in isotropic coordinates

In[∘]:=
```
rho0 = rhoCentre[RStar];
```

In[∘]:=
```
rho[rbar_] := Piecewise[
  {
   {rho0 (1 - rofrbarInside[rbar]/RStar)^6, rbar < rbarStar},
   {0, rbar ≥ rbarStar}
  }
 ];
```

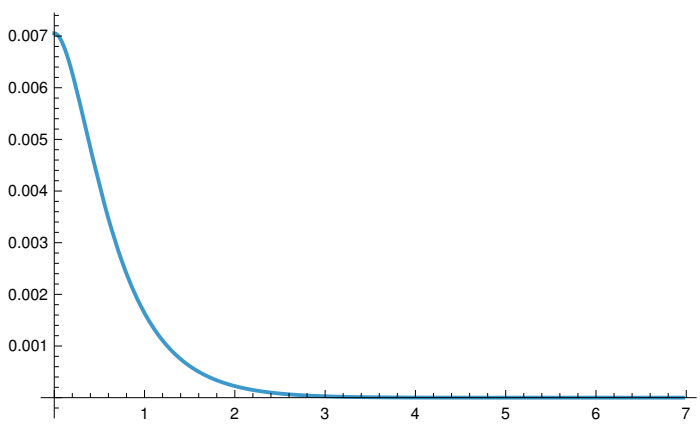
In[∘]:=
```
(*Maybe issue with NDSolve is that there are two numerically
 integrated things here being used and its causing issues?*)
(*Going to make the above as one interpolating function*)
data = Table[{rbarofrInside[r], Pressure[r]},
   {r, rmin, RStar, RStar/1000}];
```

In[∘]:=
```
IsotropicCoordsPressure = Interpolation[data];
```

In[∘]:=
```
(*Solved the issue of the solver not being able
 to get an equation for rbar'[t] (method residual error)*)
```

In[∘]:=
```
Plot[IsotropicCoordsPressure[rbar], {rbar, 0, rbarStar}, PlotRange → All]
```

Out[∘]=



Sorting out the speed of sound as a function of r: a(r)

In[∘]:=
```
(*Getting the derivative of the density*)
rhodummy[r_] := rho0 (1 - r/RStar)^6
rhodummy'[r];
drhodr[r_] := -6 rho0/RStar (1 - r/RStar)^5
```

Speed of sound in isotropic coordinates

In[∘]:=
```
cutoffIso = .97 rbarStar
```
Out[∘]=
```
6.75518
```

In[∘]:=
```
asosEndInterp[rbar_] := Module[{r = rofrbarInside[rbar]}, Sqrt[dPdr[r, Pressure[r]] / drhodr[r]]
 ]
```

In[∘]:=
```
asosEndInterp[.9987 rbarStar]
```
Out[∘]=
```
0.0000259352
```

In[∘]:=
```
data = Prepend[Table[
    {rbar, asosEndInterp[rbar]},
    {rbar, cutoffIso, .999 rbarStar, (rbarStar - cutoffIso)/10}
    (*10000 points and A is not messy*)
   ], {rbarStar, asosEndInterp[.9987 rbarStar]}];
```

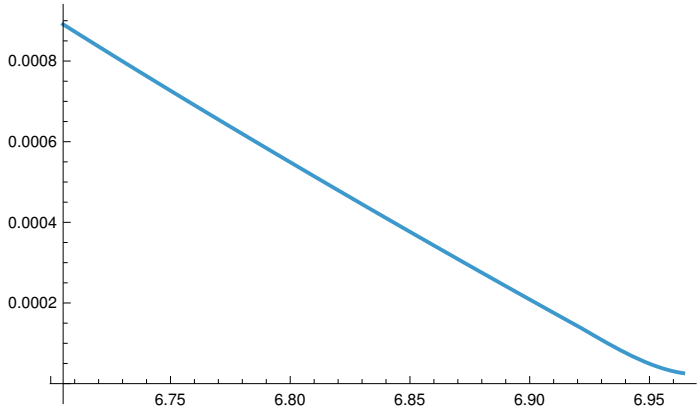
In[∘]:=
```
asosEnd = Interpolation[data]
```
Out[∘]=
```
InterpolatingFunction[ Domain: {{6.76, 6.96}} Output: scalar ]
```

In[∘]:=
```
asosPiecewise[rbar_?NumericQ] := Module[{r = rofrbarInside[rbar]}, Piecewise[{
    {Sqrt[dPdr[r, Pressure[r]] / drhodr[r]], rbar < cutoffIso},
    {asosEnd[rbar], cutoffIso ≤ rbar ≤ rbarStar},
    {asosEnd[rbarStar], rbar > rbarStar}
   }
  ]
 ]
```

In[∘]:=
```
Plot[asosEnd[rbar], {rbar, cutoffIso - 0.05, rbarStar}]
```

InterpolatingFunction: Input value {6.70518} lies outside the range of data in the interpolating function. Extrapolation will be used.

Out[∘]=



Setting up 2D ODEs for new density profile:

Constants that are going to be used

In[∘]:=
```
m = 10^-6;
```

Mach number interpolation

Interpolating

In[∘]:=
```
fMachLow[m_] := 1/2 Log[(1 + m)/(1 - m)] - m;
fMachhigh[m_] := 1/2 Log[1 - 1/m^2] + 10
```

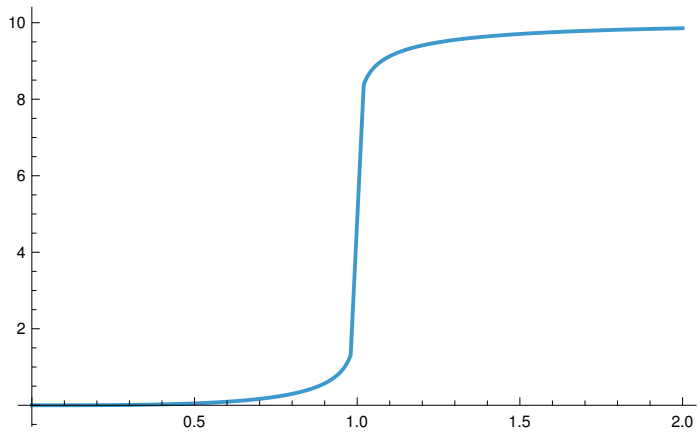
In[∘]:=
```
x1 = .98;
x2 = 1.02;
y1 = fMachLow[x1];
y2 = fMachhigh[x2];
```

In[∘]:=
```
anaylticline[Mach_] := y1 + (y2 - y1)(Mach - x1)/(x2 - x1)
```

In[∘]:=
```
IVariableInterpolated[Mach_] := Piecewise[{
    {1/2 Log[(1 + Mach)/(1 - Mach)] - Mach, Mach < 0.98},
    {anaylticline[Mach], 0.98 < Mach < 1.02},
    {1/2 Log[1 - 1/Mach^2] + 10, Mach > 1.02}
  }];
```

In[∘]:=
```
Plot[IVariableInterpolated[Mach], {Mach, 0, 2}, PlotRange → All, Exclusions → None]
```

Out[∘]=



Outside Star

Velocities

In[∘]:=
```
Ut[rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] := Sqrt[1 + (ur/A[rbar])^2 + (uphi/(rbar A[rbar]))^2]/α[rbar];

umagOutside[rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] := Sqrt[(ur^2 + (uphi/rbar)^2)]/A[rbar]
```

Rates of change of velocities

In[∘]:=
```
drBardt[rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] := 1/(A[rbar]^2 Ut[rbar, ur, uphi]) ur;

dphidtOutside[rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
  1/(rbar^2 * A[rbar]^2 * Ut[rbar, ur, uphi]) uphi;
```

Gravitational Waves

```
In[•]:= FRRr[m_?NumericQ, rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
  Module[{r = rofrbarOutside[rbar]}, ((8 m^2)/(15 r^4)*drBardt[rbar, ur, uphi]*
     (8 M^2 + r M (6 drBardt[rbar, ur, uphi]^2 + 36 r^2 dphidtOutside[rbar, ur, uphi]^2)))];

In[•]:= FRRphi[m_?NumericQ, rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
  Module[{r = rofrbarOutside[rbar]}, ((8 m^2)/(5 r^4)*dphidtOutside[rbar, ur, uphi]*
     (2 M^2 + r M (9 drBardt[rbar, ur, uphi]^2 - 6 r^2 dphidtOutside[rbar, ur, uphi]^2)))];

In[•]:= duphidtOutside[m_?NumericQ, rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
  η/m (A[rbar]^2 rbar^2 FRRphi[m, rbar, ur, uphi]);

durdt2DOutside[m_?NumericQ, rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
  -α[rbar] × Ut[rbar, ur, uphi] × DERαOutside[rbar] + ur^2/Ut[rbar, ur, uphi] (DERAOutside[rbar]/A[rbar]^3) +
   uphi^2/Ut[rbar, ur, uphi] (1/(rbar^3 * A[rbar]^2) + DERAOutside[rbar]/(rbar^2 * A[rbar]^3)) +
   η/m (A[rbar]^2 FRRr[m, rbar, ur, uphi]);
```

Inside Star

Velocities

```
In[•]:= UtInside[rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
  Sqrt[1 + (ur/AInside[rbar])^2 + (uphi/(rbar AInside[rbar]))^2] / αInside[rbar];

umagInside[rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] := Sqrt[(ur^2 + (uphi/rbar)^2)] / AInside[rbar]
```

Mach business

```
In[•]:= Mach[rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
  (Sqrt[(ur^2 + (uphi/rbar)^2)] / AInside[rbar]) / (asosPiecewise[rbar] × αInside[rbar] × UtInside[rbar, ur, uphi])
```

Dissipative Forces

Accretion

In[∘]:=
```
mdot[m_?NumericQ, rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
 4 Pi rho[rbar] m^2 / ((asosPiecewise[rbar]^2 + (umagInside[rbar, ur, uphi] / (α Inside[rbar] × UtInside[rbar, ur, uphi]))^2)^(3/2));
(*General Bondi-Hoyle-Lyttleton formula for mass accretion with q = 3 and λ=1*)
```

In[∘]:=
```
dpdtaccretionr[m_?NumericQ, rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
 -((4 Pi αInside[rbar] (rho[rbar] + IsotropicCoordsPressure[rbar]) m^2)) (1 / ((asosPiecewise[rbar]^2 + (umagInside[rbar, ur, uphi] / (α Inside[rbar] × UtInside[rbar, ur, uphi]))^2)^(3/2))) ur;

dpdtaccretionphi[m_?NumericQ, rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
 -((4 Pi αInside[rbar] (rho[rbar] + IsotropicCoordsPressure[rbar]) m^2)) (1 / ((asosPiecewise[rbar]^2 + (umagInside[rbar, ur, uphi] / (α Inside[rbar] × UtInside[rbar, ur, uphi]))^2)^(3/2))) uphi
```

Drag

In[∘]:=
```
dpdtdragphi[m_?NumericQ, rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
 1 / umagInside[rbar, ur, uphi]^3 (-4 π αInside[rbar] *
    (rho[rbar] + IsotropicCoordsPressure[rbar]) * m^2 * (1 + 2 umagInside[rbar, ur, uphi]^2)^2 *
    IVariableInterpolated[Mach[rbar, ur, uphi]] * uphi);

dpdtdragr[m_?NumericQ, rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
 (-4 π αInside[rbar] (rho[rbar] + IsotropicCoordsPressure[rbar])
    m^2 (1 + 2 umagInside[rbar, ur, uphi]^2)^2
    IVariableInterpolated[Mach[rbar, ur, uphi]] * ur) / umagInside[rbar, ur, uphi]^3;
```

Rates of change of velocities

In[∘]:=
```
drBardtInside[rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
  1/(AInside[rbar]^2 UtInside[rbar, ur, uphi]) ur;

dphidtInside[rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
  1/(rbar^2 AInside[rbar]^2 UtInside[rbar, ur, uphi]) uphi;
```

In[∘]:=
```
(*Redefining the mass function without the piecewise component for simplicity*)
MassFunction[r_] :=
  252 (r^9/(9 RStar^9) - 3 r^8/(4 RStar^8) + 15 r^7/(7 RStar^7) - 10 r^6/(3 RStar^6) + 3 r^5/RStar^5 - 3 r^4/(2 RStar^4) + r^3/(3 RStar^3));
```

In[∘]:=
```
Mp[r_] := Derivative[1][MassFunction][r]
Mpp[r_] := Derivative[2][MassFunction][r]
Mppp[r_] := Derivative[3][MassFunction][r]
```

In[∘]:=
```
FRRrIns[m_?NumericQ, rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
  Module[{r = rofrbarInside[rbar]}, (8 m^2)/(15 r^4) * drBardtInside[rbar, ur, uphi] *
    (8 MassFunction[r]^2 + r MassFunction[r] (6 drBardtInside[rbar, ur, uphi]^2 +
          36 r^2 dphidtInside[rbar, ur, uphi]^2 + Mp[r] - 3 r Mpp[r]) -
      r^2 (Mp[r]^2 + 3 r^2 dphidtInside[rbar, ur, uphi]^2 (6 Mp[r] - r Mpp[r]) +
          drBardtInside[rbar, ur, uphi]^2 (2 Mp[r] + r Mpp[r] - r^2 Mppp[r])))];
```

In[∘]:=
```
FRRphiIns[m_?NumericQ, rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
  Module[{r = rofrbarInside[rbar]}, (8 m^2)/(5 r^4) * dphidtInside[rbar, ur, uphi] *
    (2 MassFunction[r]^2 + 2 r^4 dphidtInside[rbar, ur, uphi]^2 × Mp[r] + r MassFunction[r]
        (9 drBardtInside[rbar, ur, uphi]^2 - 6 r^2 dphidtInside[rbar, ur, uphi]^2 - 2 Mp[r]) -
      r^2 drBardtInside[rbar, ur, uphi]^2 (7 Mp[r] - 2 r Mpp[r]))];
```

In[∘]:=
```
duphidtInside[m_?NumericQ, rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
  η/m (dpdtaccretionphi[m, rbar, ur, uphi] + dpdtdragphi[m, rbar, ur, uphi] +
     AInside[rbar]^2 rbar^2 FRRphiIns[m, rbar, ur, uphi]);

durdt2DInside[m_?NumericQ, rbar_?NumericQ, ur_?NumericQ, uphi_?NumericQ] :=
  -αInside[rbar] × UtInside[rbar, ur, uphi] × DERαInside[rbar] +
   ur^2/UtInside[rbar, ur, uphi] (DERAInside[rbar]/AInside[rbar]^3) +
   uphi^2/UtInside[rbar, ur, uphi] (1/(rbar^3 AInside[rbar]^2) + DERAInside[rbar]/(rbar^2 AInside[rbar]^3)) +
   η/m (dpdtaccretionr[m, rbar, ur, uphi] + dpdtdragr[m, rbar, ur, uphi] +
     AInside[rbar]^2 FRRrIns[m, rbar, ur, uphi]);
```

Defining Energy and Angular momentum per unit mass functions along with other orbital parameters

In[∘]:=
```
EnergyPerUnitMass[ptilde_, eccen_] := Sqrt[((ptilde - 2)^2 - 4 * eccen^2)/(ptilde * (ptilde - 3 - eccen^2))];

LPerUnitMass[ptilde_, M_, eccen_] := (ptilde * M)/Sqrt[ptilde - 3 - eccen^2];

ptildefunc[p_, Mass_] := p/Mass;

rapoapsis[eccen_, ptilde_, Mass_] := ((ptilde) * Mass)/(1 - eccen);

(*In Schwarzschild areal r coordinates*)
rperiapsis[eccen_, ptilde_] := ptilde/(1 + eccen);
```

Defining the constants to define the motion

Eccentricity and semi-latus rectum

In[∘]:=
```
eccentricity = 0.4;
(*eccentricity = 9/11; (*Obtained from setting periapsis to 5.5 and ptilde still to 10*)*)
semilatrecP = 10;
ptilde = ptildefunc[semilatrecP, M];
```

Energy and Angular momentum per unit mass

In[∘]:=
```
Energy = EnergyPerUnitMass[ptilde, eccentricity];
AngularMomentumUphi0 = LPerUnitMass[ptilde, M, eccentricity];
```

Initial drop radius (apoapsis), closest point it goes to (perihelion), defining Isotropic star radius

```
r0 = rapoapsis[eccentricity, ptilde, M];
rbar0 = rbarofrOutside[r0];

rperiapsis[eccentricity, ptilde];
```

Must satisfy this inequality to pass through star

In[∘]:=
```
ptilde < RStar (1 + eccentricity)
ptilde > 6 + 2 eccentricity
```

Out[∘]= True

Out[∘]= True

Solving the ODEs

In[∘]:=
```
drBardtInside[rbarStar, 0, AngularMomentumUphi0]
durdt2DInside[m, rbarStar, 0, AngularMomentumUphi0]
duphidtInside[m, rbarStar, 0, AngularMomentumUphi0]
dphidtInside[rbarStar, 0, AngularMomentumUphi0]
```

Out[∘]= 0.

InterpolatingFunction: Input value $\left\{\frac{1}{2}\left(7+4\sqrt{3}\right)\right\}$ lies outside the range of data in the interpolating function. Extrapolation will be used.

Out[∘]= 0.00219943

Out[∘]= $-5.4786\times10^{-9}\,\eta$

Out[∘]= 0.0466817

```
t0 = 0;
tEnd = 10 000;
η = 1.5 × 10^4;

Monitor[
  solIn = NDSolve[
    {
```

```
rbar'[t] ==
 Piecewise[
  {
   {drBardt[rbar[t], ur[t], uphi[t]], rbar[t] ≥ rbarStar},
   {drBardtInside[rbar[t], ur[t], uphi[t]], rbar[t] < rbarStar}
  }
 ],
ur'[t] ==
 Piecewise[
  {
   {durdt2DOutside[mass[t], rbar[t], ur[t], uphi[t]], rbar[t] ≥ rbarStar},
   {durdt2DInside[mass[t], rbar[t], ur[t], uphi[t]], rbar[t] < rbarStar}
  }
 ],
uphi'[t] ==
 Piecewise[
  {
   {duphidtOutside[mass[t], rbar[t], ur[t], uphi[t]], rbar[t] ≥ rbarStar},
   {duphidtInside[mass[t], rbar[t], ur[t], uphi[t]], rbar[t] < rbarStar}
  }
 ],
phi'[t] ==
 Piecewise[
  {
   {dphidtOutside[rbar[t], ur[t], uphi[t]], rbar[t] ≥ rbarStar},
   {dphidtInside[rbar[t], ur[t], uphi[t]], rbar[t] < rbarStar}
  }
 ],
mass'[t] ==
 Piecewise[
  {
   {0, rbar[t] ≥ rbarStar},
   {mdot[mass[t], rbar[t], ur[t], uphi[t]], rbar[t] < rbarStar}
  }
 ],
rbar[t0] == rbar0,
ur[t0] == 0,
phi[t0] == 0,
uphi[t0] == AngularMomentumUphi0,
mass[t0] == m,

WhenEvent[rbar[t] ≤ 10^-2 , "StopIntegration"]
```

```
      },
      {rbar, ur, phi, uphi, mass},
      {t, t0, tEnd},
      MaxSteps → Infinity,
      WorkingPrecision → 30,

      StepMonitor :→ (currentTime = t)(*Machine precision and 20k max steps also works*)
     ][[1]],
   Column[{
     Row[{"Progress: ", ProgressIndicator[currentTime, {t0, tEnd}]}],
     Row[{"Current Time: ", NumberForm[currentTime, {6, 2}], " / ", tEnd}]
     (*Amount of digits displayed, amount of decimals rounded*)
    }]
  ];
```

NDSolve: The precision of the differential equation

$$\left(\left\{\left\{\text{rbar}'[t] == \begin{cases} \text{drBardt[rbar[≪1≫], ur[≪1≫], uphi[≪1≫]} & \text{rbar[≪1≫]} \geq \text{Times[≪2≫]} \\ \text{drBardtInside[rbar[≪1≫], ur[≪1≫], uphi[≪1≫]]} & \text{rbar[≪1≫]} < \text{Times[≪2≫]} \\ 0 & \text{True} \end{cases}, \text{ur}'[t] == \begin{cases} \text{durdt2D(} \\ \text{durdt2DI} \\ 0 \end{cases}\right.\right.\right.$$

$$== \begin{cases} 0 & \text{rbar[≪1≫]} \geq \text{Times[≪2≫]} \\ \text{mdot[mass[≪1≫], rbar[≪1≫], ur[≪1≫], uphi[≪1≫]]} & \text{rbar[≪1≫]} < \text{Times[≪2≫]} \\ 0 & \text{True} \end{cases}, \text{rbar}[0] ==$$

$$15.6507, \text{ur}[0] == 0, \text{phi}[0] == 0, \text{uphi}[0] == 3.8236, \text{mass}[0] == \frac{1}{1000000}\Big\}, \{\}, \{\}, \{\}, \Big\{\text{WhenEvent}\Big[\text{rbar}[t] \leq$$

$$\left.\left.\left.\frac{1}{10^2}, \text{StopIntegration}\Big]\Big\}\right\}\right) \text{ is less than WorkingPrecision (30.`).}$$

InterpolatingFunction: Input value {6.964101615137754587054892683 01} lies outside the range of data in the interpolating function. Extrapolation will be used.

InterpolatingFunction: Input value {6.964101615137754587054511622 36} lies outside the range of data in the interpolating function. Extrapolation will be used.

InterpolatingFunction: Input value {6.964101297910252961223127360 11} lies outside the range of data in the interpolating function. Extrapolation will be used.

General: Further output of InterpolatingFunction::dmval will be suppressed during this calculation.

```
Abs[6.964101615137754587054511622362634103295243207361591332781 5 - rbarStar]*
 10 000 000 000 (*Error is miniscule now on every entry :))))))))*)
```

Out[•]= $3.8106064911063059036730025916992333 0 \times 10^{-12}$

In[•]:=
```
rbarNumIn = rbar /. solIn;
urNumIn = ur /. solIn;
phiNumIn = phi /. solIn;
uphiNumIn = uphi /. solIn;
massNumIn = mass /. solIn;
```

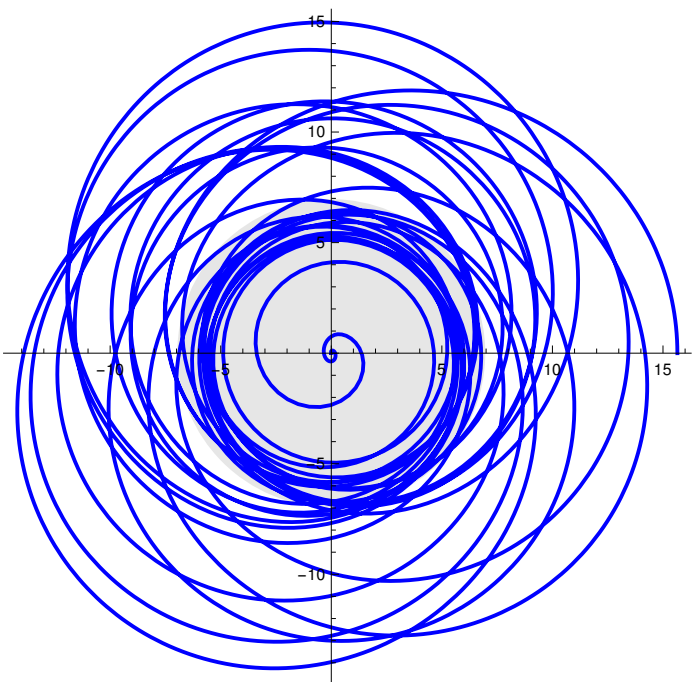
In[∘]:=
```
tFinish = rbarNumIn["Domain"][[1, 2]]
tSurface = t /. FindRoot[rbarNumIn[t] == rbarStar, {t, 10, 10, tFinish}]
```

Out[∘]= 4971.766518110853961774021747 43

Out[∘]= 159.739

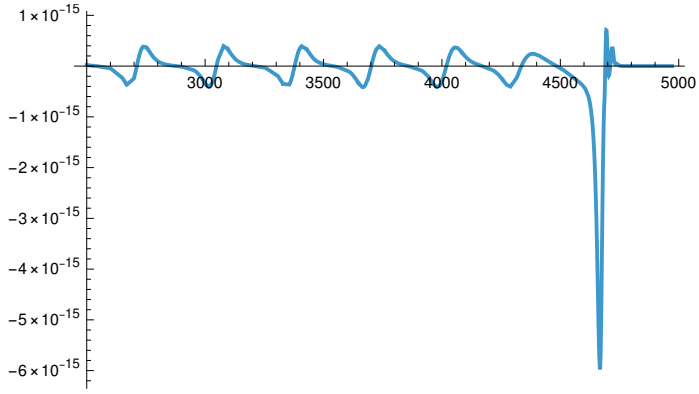
In[∘]:=
```
x[t_] := rbarNumIn[t] Cos[phiNumIn[t]];
y[t_] := rbarNumIn[t] Sin[phiNumIn[t]];
orbit2D = ParametricPlot[{x[t], y[t]}, {t, t0, 0.999 tFinish}, AspectRatio → Automatic,
   AxesLabel → {"x", "y"}, PlotRange → All, PlotStyle → {Thick, Blue}];
(*Automatic scale makes the aces the same scale*)
centralBody = Graphics[{GrayLevel[0.9], Disk[{0, 0}, rbarStar]}];
Show[centralBody, orbit2D, AspectRatio → Automatic, PlotRange → All, Axes → True]
```

Out[∘]=



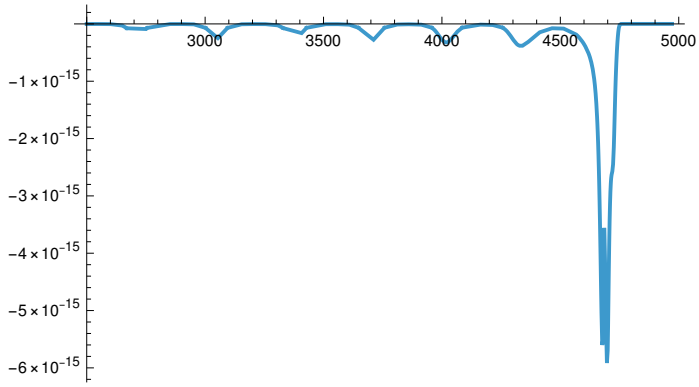
In[∘]:=
```
FRRrFull[m_, rbar_, ur_, uphi_] := Piecewise[
  {
   {FRRr[m, rbar, ur, uphi], rbar ≥ rbarStar},
   {FRRrIns[m, rbar, ur, uphi], rbar < rbarStar}
  }
 ]
```

In[∘]:=
```
Plot[FRRrFull[massNumIn[t], rbarNumIn[t], urNumIn[t], uphiNumIn[t]],
 {t, 2500, tFinish}, PlotRange → All]
```

Out[∘]=


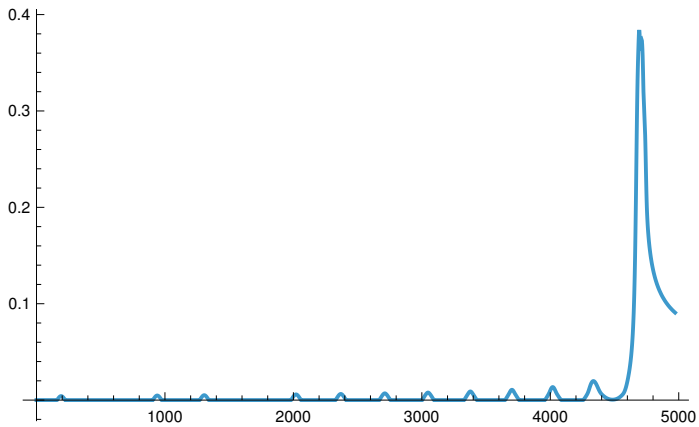
In[∘]:=
```
FRRphiFull[m_, rbar_, ur_, uphi_] := Piecewise[
  {
   {FRRphi[m, rbar, ur, uphi], rbar ≥ rbarStar},
   {FRRphiIns[m, rbar, ur, uphi], rbar < rbarStar}
  }
 ]
```

In[∘]:=
```
Plot[FRRphiFull[massNumIn[t], rbarNumIn[t], urNumIn[t], uphiNumIn[t]],
 {t, 2500, tFinish}, PlotRange → All]
```

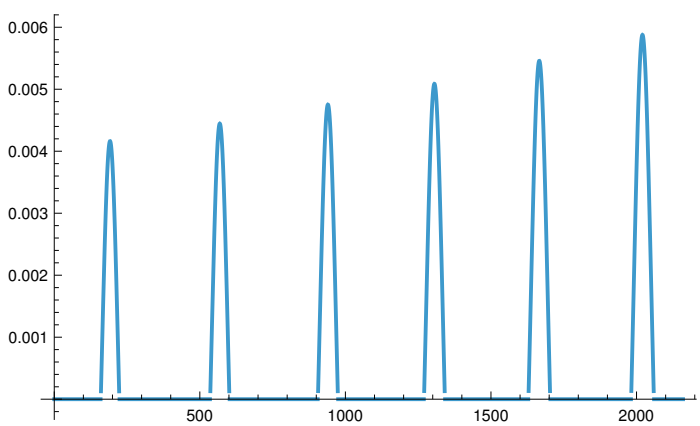
Out[∘]=


In[∘]:=
```
Plot[asosPiecewise[rbarNumIn[t]], {t, 0, tFinish}, PlotRange → All]
```

InterpolatingFunction: Input value {15.6507} lies outside the range of data in the interpolating function. Extrapolation will be used.

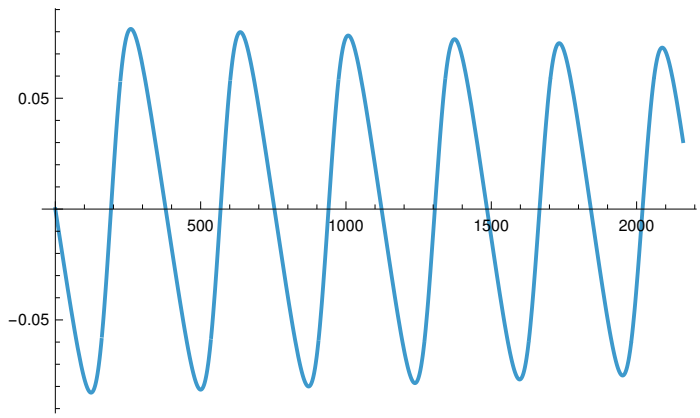
Out[∘]=


In[∘]:=
```
Plot[
 {
  Piecewise[
   {
    {0, rbarNumIn[t] ≥ rbarStar},
    {asosPiecewise[rbarNumIn[t]], rbarNumIn[t] < rbarStar}
   }
  ]
 },
 {t, 0, tSurface + 2000}, PlotRange → All
]
```

Out[∘]=


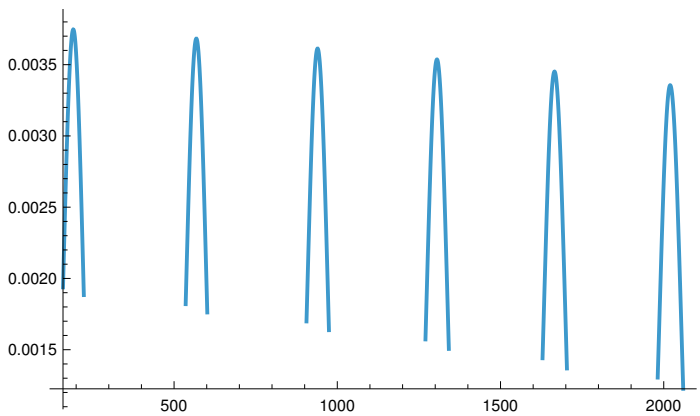
In[∘]:=
```
Plot[
 {
  Piecewise[
   {
    {drBardt[rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] ≥ rbarStar},
    {drBardtInside[rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] < rbarStar}
   }
  ]
 },
 {t, 0, tSurface + 2000}, PlotRange → All
]
```

Out[∘]=



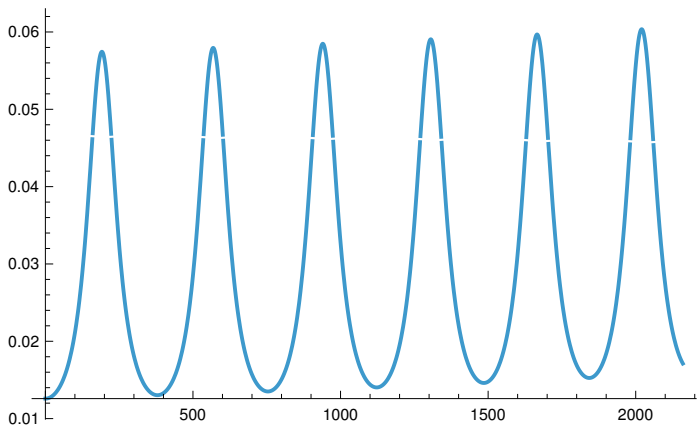
In[∘]:=
```
Plot[
 {
  Piecewise[
   {
    {durdt2DOutside[rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] ≥ rbarStar},
    {durdt2DInside[massNumIn[t],
       rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] < rbarStar}
   }
  ]
 },
 {t, 0, tSurface + 2000}, PlotRange → All
]
```

Out[∘]=

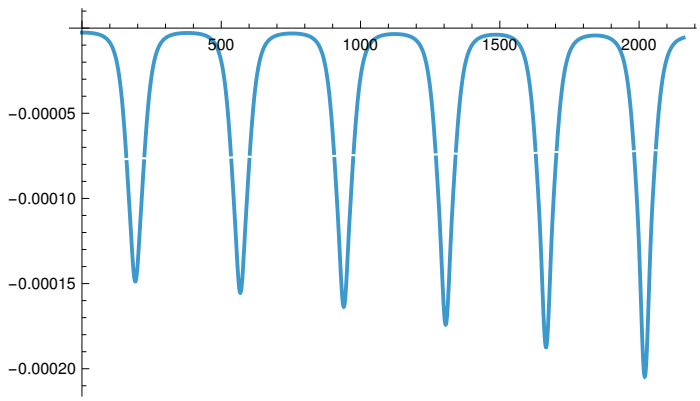


In[∘]:=
```
Plot[
 {
  Piecewise[
   {
    {dphidtOutside[rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] ≥ rbarStar},
    {dphidtInside[rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] < rbarStar}
   }
  ]
 },
 {t, 0, tSurface + 2000}, PlotRange → All
]
```

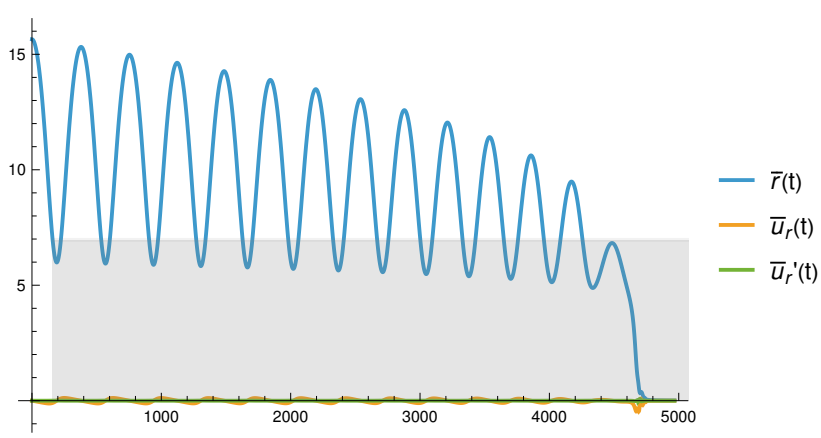
Out[∘]=



In[∘]:=
```
Plot[
 {
  Piecewise[
   {
    {duphidtOutside[massNumIn[t], rbarNumIn[t], urNumIn[t], uphiNumIn[t]],
     rbarNumIn[t] ≥ rbarStar},
    {duphidtInside[massNumIn[t],
       rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] < rbarStar}
   }
  ]
 },
 {t, 0, tSurface + 2000}, PlotRange → All
]
```

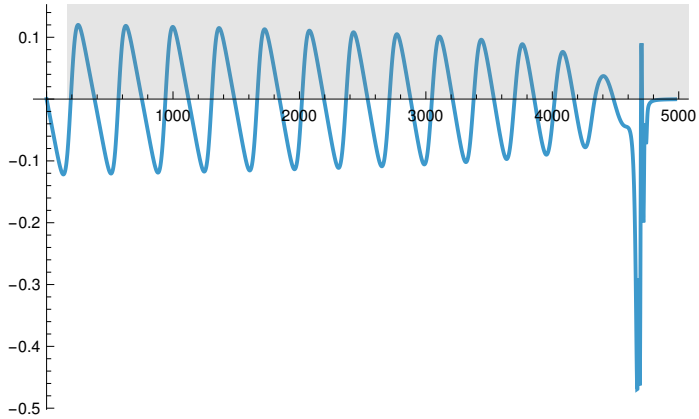
Out[∘]=


In[∘]:=
```
outFigRadial = Show[Plot[{rbarNumIn[t], urNumIn[t], urNumIn'[t]},
    {t, t0, .999 tFinish}, PlotLegends → LineLegend[Automatic, {"r̄(t)", "ū_r(t)", "ū_r'(t)"}]],
   Plot[rbarStar, {t, tSurface, tSurface + tFinish},
    PlotStyle → Directive[Gray, Opacity[0.1]], Filling → Bottom]]
```

Out[∘]=


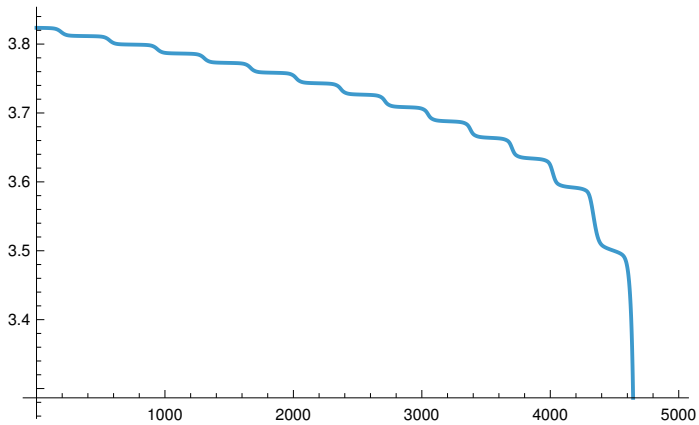
In[∘]:=
```
outFigRadial = Show[Plot[{urNumIn[t]}, {t, t0, tFinish},
    PlotLegends → LineLegend[Automatic, {"r̄(t)", "ūr(t)", "ūr'(t)"}]],
  Plot[rbarStar, {t, tSurface, tSurface + tFinish},
    PlotStyle → Directive[Gray, Opacity[0.1]], Filling → Bottom]]
```

Out[∘]=



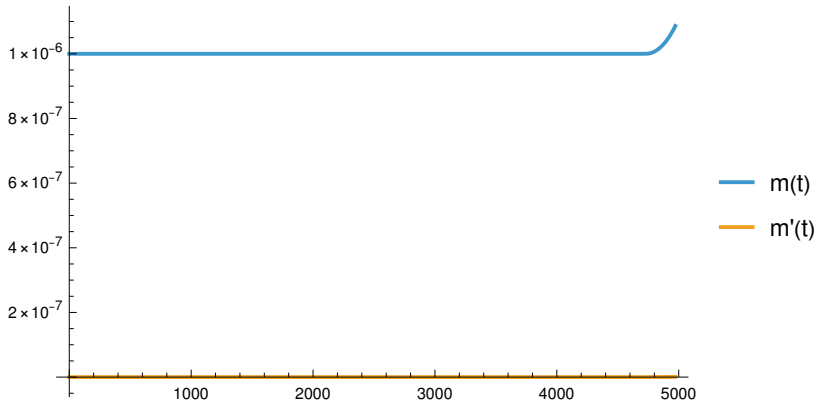
In[∘]:=
```
outFigAngular = Plot[{uphiNumIn[t]}, {t, t0, tFinish},
  PlotLegends → LineLegend[Automatic, {"phi(t)", "uphi(t)", "uphi'(t)"}]]
```

Out[∘]=



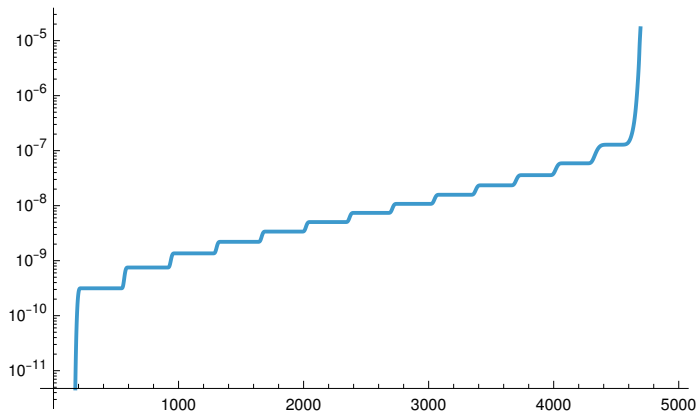
In[∘]:=
```
outFigMass = Show[Plot[{massNumIn[t], massNumIn'[t]},
    {t, t0, tFinish}, PlotLegends → LineLegend[Automatic, {"m(t)", "m'(t)"}]]]
```

Out[∘]=



In[∘]:=
```
LogPlot[Abs[1 - massNumIn[t]/massNumIn[0]], {t, 0, tFinish}]
```

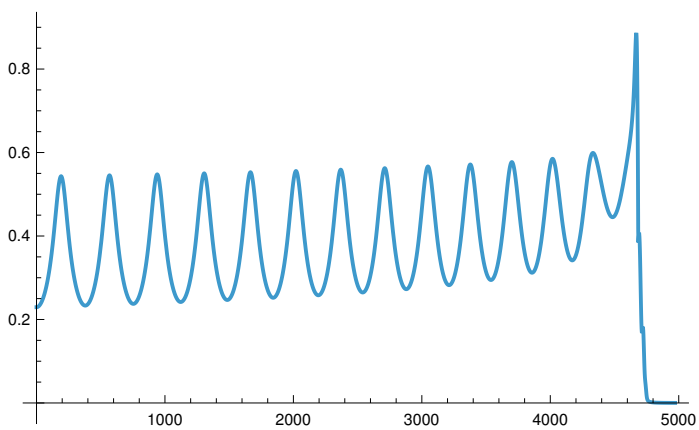
Out[∘]=



In[∘]:=
```
umag[rbar_, ur_, uphi_] := Piecewise[
  {
   {umagOutside[rbar, ur, uphi], rbar ≥ rbarStar},
   {umagInside[rbar, ur, uphi], rbar < rbarStar}
  }
 ]
```

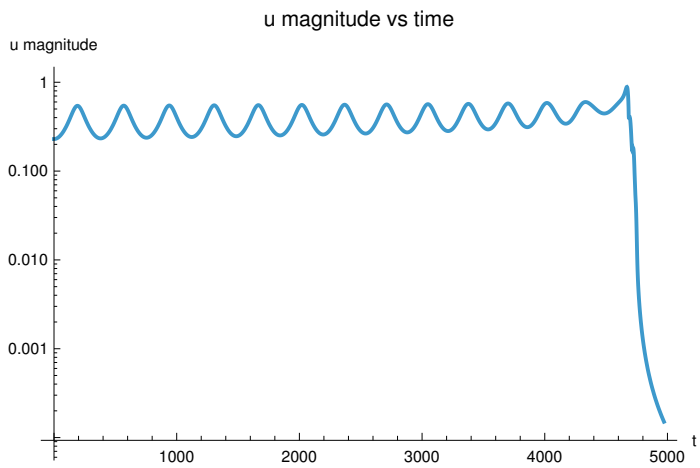
In[∘]:=
```
Plot[umag[rbarNumIn[t], urNumIn[t], uphiNumIn[t]], {t, 0, tFinish}, Exclusions → None]
(*Normal behaviour judging from rho=const simulation*)
```

Out[∘]=



In[∘]:=
```
LogPlot[Evaluate[umag[rbarNumIn[t], urNumIn[t], uphiNumIn[t]] /. solIn], {t, 0, tFinish},
 PlotRange → All, AxesLabel → {"t", "u magnitude"}, PlotLabel → "u magnitude vs time"]
```

Out[∘]=



In[∘]:=
```
(*Show me the value of u at tfinish*)
umag[rbarNumIn[tFinish], urNumIn[tFinish], uphiNumIn[tFinish]] /. solIn
```

Out[∘]= 0.000150755468427773499291512

Gravitational-wave strains

h_+

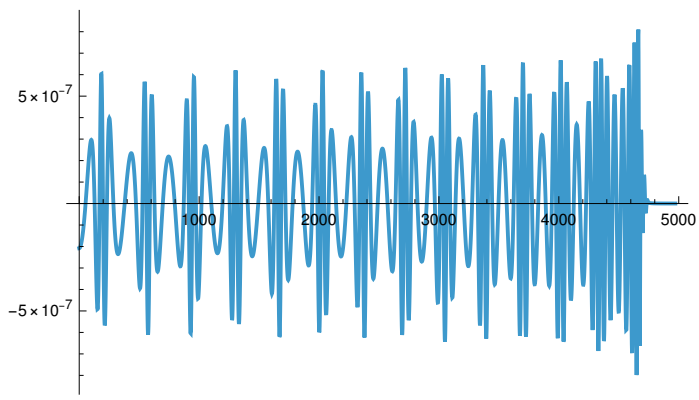
In[∘]:=
```
hplusInside[d_, m_, rbar_, ur_, uphi_, phi_] :=
 Module[{r = rofrbarInside[rbar], rd, phid},
  rd = drBardtInside[rbar, ur, uphi];
  phid = dphidtInside[rbar, ur, uphi];
  (-2 m/d) (MassFunc[r] Cos[2 phi]/r - rd^2 Cos[2 phi] + 2 rd r phid Sin[2 phi] + (r phid)^2 Cos[2 phi])
 ]
```

In[∘]:=
```
hplusOutside[d_, m_, rbar_, ur_, uphi_, phi_] :=
 Module[{r = rofrbarOutside[rbar], rd, phid},
  rd = drBardt[rbar, ur, uphi];
  phid = dphidtOutside[rbar, ur, uphi];
  (-2 m/d) (M Cos[2 phi]/r - rd^2 Cos[2 phi] + 2 rd r phid Sin[2 phi] + (r phid)^2 Cos[2 phi])
 ]
```

In[∘]:=
```
hplusFull[d_, m_, rbar_, ur_, uphi_, phi_] := Piecewise[
  {
   {hplusOutside[d, m, rbar, ur, uphi, phi], rbar ≥ rbarStar},
   {hplusInside[d, m, rbar, ur, uphi, phi], rbar < rbarStar}
  }
 ]
```

In[∘]:=
```
Plot[hplusFull[1, massNumIn[t], rbarNumIn[t], urNumIn[t], uphiNumIn[t], phiNumIn[t]],
 {t, 0, tFinish}, PlotRange → Full, Exclusions → None](*10^8 it gets jagged*)
```

Out[∘]=



h_x

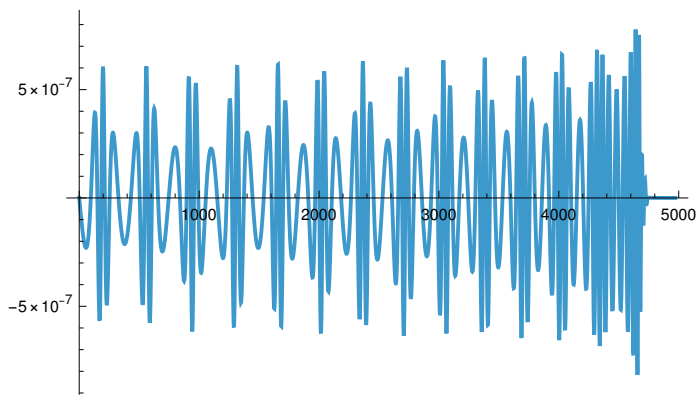
In[∘]:=
```
hxInside[d_, m_, rbar_, ur_, uphi_, phi_] := Module[{r = rofrbarInside[rbar], rd, phid},
  rd = drBardtInside[rbar, ur, uphi];
  phid = dphidtInside[rbar, ur, uphi];
  (-2 m/d) ((MassFunc[r] Sin[2 phi]/r - 2 (rd Sin[phi] + r phid Cos[phi]) (rd Cos[phi] - r phid Sin[phi]))
 ]
```

In[∘]:=
```
hxOutside[d_, m_, rbar_, ur_, uphi_, phi_] := Module[{r = rofrbarOutside[rbar], rd, phid},
  rd = drBardt[rbar, ur, uphi];
  phid = dphidtOutside[rbar, ur, uphi];
  (-2 m/d) ((M Sin[2 phi]/r - 2 (rd Sin[phi] + r phid Cos[phi]) (rd Cos[phi] - r phid Sin[phi]))
 ]
```

In[∘]:=
```
hxFull[d_, m_, rbar_, ur_, uphi_, phi_] := Piecewise[
  {
   {hxOutside[d, m, rbar, ur, uphi, phi], rbar ≥ rbarStar},
   {hxInside[d, m, rbar, ur, uphi, phi], rbar < rbarStar}
  }
 ]
```

In[∘]:=
```
Plot[hxFull[1, massNumIn[t], rbarNumIn[t], urNumIn[t], uphiNumIn[t], phiNumIn[t]],
 {t, 0, tFinish}, PlotRange → Full, Exclusions → None](*10^8 it gets jagged*)
```

Out[∘]=



Energy loss during orbits **CHECK angles and other assumptions made by authors for these formulae especially the angular momentum stuff.**

Instantaneous energy loss (coordinate system dependent)

In[∘]:=
```
dEdtInside[m_, rbar_, ur_, uphi_] :=
 Module[{r = rofrbarInside[rbar], rd, phid}, rd = drBardtInside[rbar, ur, uphi];
  phid = dphidtInside[rbar, ur, uphi];
  (8 m^2)/(15 r^4)*(MassFunction[r]^2*(8 rd^2 + 6 (r phid)^2) + r MassFunction[r]*
      (6 rd^4 - 6 (3 (r phid)^4 + (r phid)^2 Mp[r]) + rd^2 (63 (r phid)^2 + Mp[r] - 3 r Mpp[r])) +
     r^2*(6 (r phid)^4 Mp[r] - rd^2 ((Mp[r])^2 + 3 (r phid)^2 (13 Mp[r] - 3 r Mpp[r])) -
        rd^4 (2 Mp[r] + r Mpp[r] - r^2 Mppp[r])))]
```

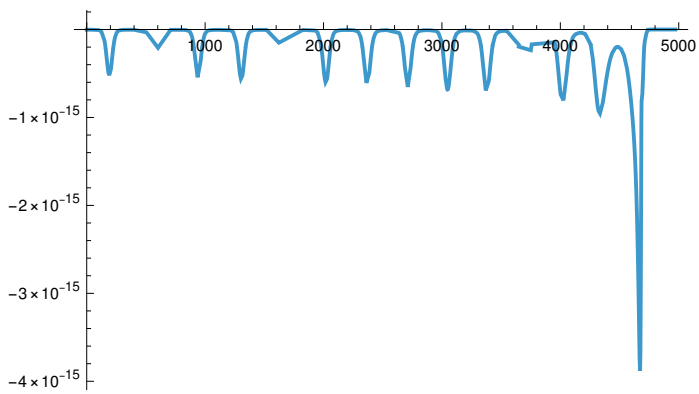
In[∘]:=
```
dEdtOutside[m_, rbar_, ur_, uphi_] :=
 Module[{r = rofrbarOutside[rbar], rd, phid}, rd = drBardt[rbar, ur, uphi];
  phid = dphidtOutside[rbar, ur, uphi];
  (8 m^2)/(15 r^4)*
   (M^2*(8 rd^2 + 6 (r phid)^2) + r M*(6 rd^4 - 6 (3 (r phid)^4) + rd^2 (63 (r phid)^2)))]
```

In[∘]:=
```
dEdtFull[m_, rbar_, ur_, uphi_] := Piecewise[
  {
   {dEdtOutside[m, rbar, ur, uphi], rbar ≥ rbarStar},
   {dEdtInside[m, rbar, ur, uphi], rbar < rbarStar}
  }
 ]
```

In[∘]:=
```
Plot[dEdtFull[massNumIn[t], rbarNumIn[t], urNumIn[t], uphiNumIn[t]],
 {t, 0, tFinish}, PlotRange → All]
```

InterpolatingFunction: Input value {15.6507} lies outside the range of data in the interpolating function. Extrapolation will be used.

Out[∘]=



Average, physical energy loss per orbit

In[∘]:=
```
dEdtAvg[m_, M_, a_, e_] := -(32 m ^ 2 M ^ 3) / (5 a ^ 5 (1 - e ^ 2) ^ (7 / 2)) * (1 + (73 / 24) e ^ 2 + (37 / 96) e ^ 4)
```

Need formulae for M[t]? Although accretion is very small. a(t) and e(t)

Angular momentum loss during orbits

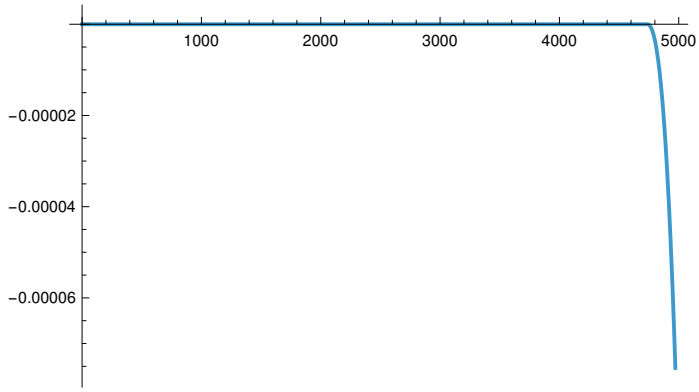
Instantaneous angular momentum loss (coordinate system dependent)

In[∘]:=
```
dLdt[m_, M_, a_, e_, phi_] :=
```

$$-\frac{4\left(m^2 M^{\frac{5}{2}}\right)}{5\left(a^{\frac{7}{2}}\left(1-e^2\right)^{\frac{7}{2}}\right)}\left(1+e\,\mathrm{Cos}[\mathrm{phi}]\right)^3\left(8-3e^2+20\,e\,\mathrm{Cos}[\mathrm{phi}]+15\,e^2\,\mathrm{Cos}[2\,\mathrm{phi}]\right)$$

In[∘]:=
```
Plot[dLdt[massNumIn[t], M, rbarNumIn[t], urNumIn[t], uphiNumIn[t]],
 {t, 0, tFinish}, PlotRange → All]
```

Out[∘]=



Average, physical angular momentum loss per orbit

In[∘]:=
```
(*Formula might need to be adjusted for me as they use the angle phi =
 0 for when it crosses the star surface which is not true for me*)
```

In[∘]:=
```
dLdtAvg[m_, M_, a_, e_] := -(32 m ^ 2 M ^ (5 / 2)) / (5 a ^ (7 / 2) (1 - e ^ 2) ^ (2)) * (1 + (7 / 8) e ^ 2)
```

Need formulae for M[t]? Although accretion is very small. a(t) and e(t).

Use E and L to find a(t) and e(t)??